1 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ

Тип приложения: веб-приложение с микросервисной архитектурой.

Стратегия развёртывания:

1 Контейнеризация: использование Docker для упаковки приложения и его зависимостей.

2 Оркестрация: использование Docker Compose(разработка) и Kubernetes(продакшен).

3 База данных: индивидуальные базы данных PostgreSQL для каждого микросервиса.

Обоснование выбора технологии:

1 Backend: Java Spring Boot – фреймворк с поддержкой ORM (Spring Data JPA), JWT(Spring Security), микросервисной архитектуры(Spring Cloud).

2 Frontend: JavaScript, React — стандарт для веб-интерфейсов, кроссплатформенность.

3 База данных: PostgreSQL – обеспечивает выполнение ACID, интеграцию с Spring Boot.

Показатели качество

Показатели качества

1 Функциональность:

– микросервисное взаимодействие;

– CRUD-операции для профилей пользователей и предметов аренды;

– валидация данных на уровне моделей и DTO.

2 Производительность:

– время отклика сервера < 200 мс для основных операций;

– оптимизация запросов в БД через встроенную библиотеку HikariCP.

3 Масштабируемость:

– возможность запуска нескольких экземпляров микросервисов под управлением балансировщика без потери состояния сессии (Stateless);

– возможность добавления новых микросервисов без изменения логики существующих.

4 Безопасность:

– использование JWT с механизмом Refresh Token.

– хранение хэшей паролей, полученных с использованием алгоритма BCrypt.

5 Удобство использования:

– информативность JSON-ответов с понятными кодами ошибок;

– предоставление Swagger для тестирования методов.

Пути реализации сквозной функциональности:

1 Аутентификация и авторизация: реализуется через Spring Security и фильтры для валидации JWT. Разграничение прав доступа на уровне методов контроллера (аннотации @PreAuthorize).

2 Логирование: настройка централизованного логирования с использованием SLF4J/Logback.

3 Обработка ошибок: глобальная обработка исключений с помощью @ControllerAdvice, которая преобразует внутренние ошибки сервера в читаемые пользователем сообщения в формате JSON.

4 Кеширование: применение Spring Cache (интеграция с Redis) для хранения часто запрашиваемых данных.

5 API Документация: Автоматическая генерация интерактивной документации через Springdoc-openapi (Swagger UI), доступной по адресу /swagger-ui.html.

Структурная схема приложения представлена на рисунке 1.1.

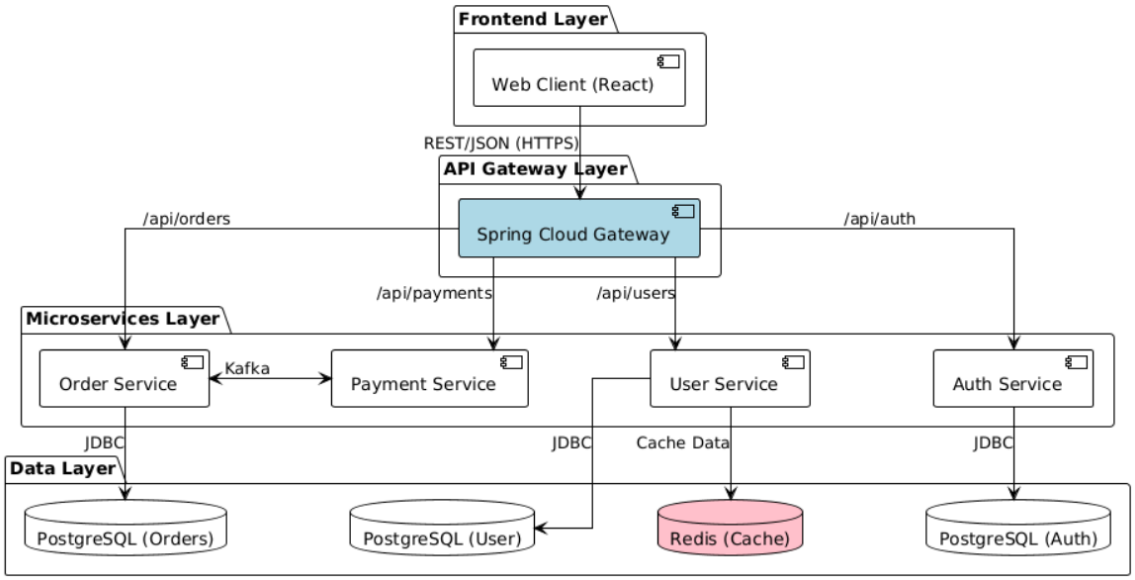


Рисунок 1.1 – Архитектура “To Be”

Архитектурные слои:

1 Уровень клиента/фронтэнда(Frontend Layer):

– компоненты: веб-интерфейс (JS/React);

– функция: Отправка HTTP-запросов в формате JSON. Клиент не знает о внутреннем устройстве системы и взаимодействует только с единой точкой входа.

2 Уровень шлюза (API Gateway Layer):

– технология: Spring Cloud Gateway;

– роль: «Единая точка входа» (Entry Point);

– задачи:

а) маршрутизация: направляет запрос к нужному сервису;

б) безопасность: первичная проверка валидности JWT-токена;

в) лимитирование: ограничение частоты запросов.

3 Уровень бизнес-логики (Microservices Layer):

– Auth Service: регистрация, аутентификация, выдача JWT/Refresh токенов;

– User Service: управление профилями, рейтингами и контактными данными;

– Order Service: управление каталогом вещей, бронированием и жизненным циклом сделки;

– Payment Service: взаимодействие с внешними платежными шлюзами и фиксация транзакций.

4 Уровень данных (Data Layer):

– PostgreSQL: основное хранилище. Используется паттерн Database-per-Service (у каждого сервиса своя схема/БД) для обеспечения независимости;

– Redis: слой кеширования и хранения быстрых данных.

2 АНАЛИЗ АРХИТЕКТУРЫ

Диаграмма классов на основании кода первого спринта в приложении А.

3 СРАВНЕНИЕ И РЕФАКТОРИНГ

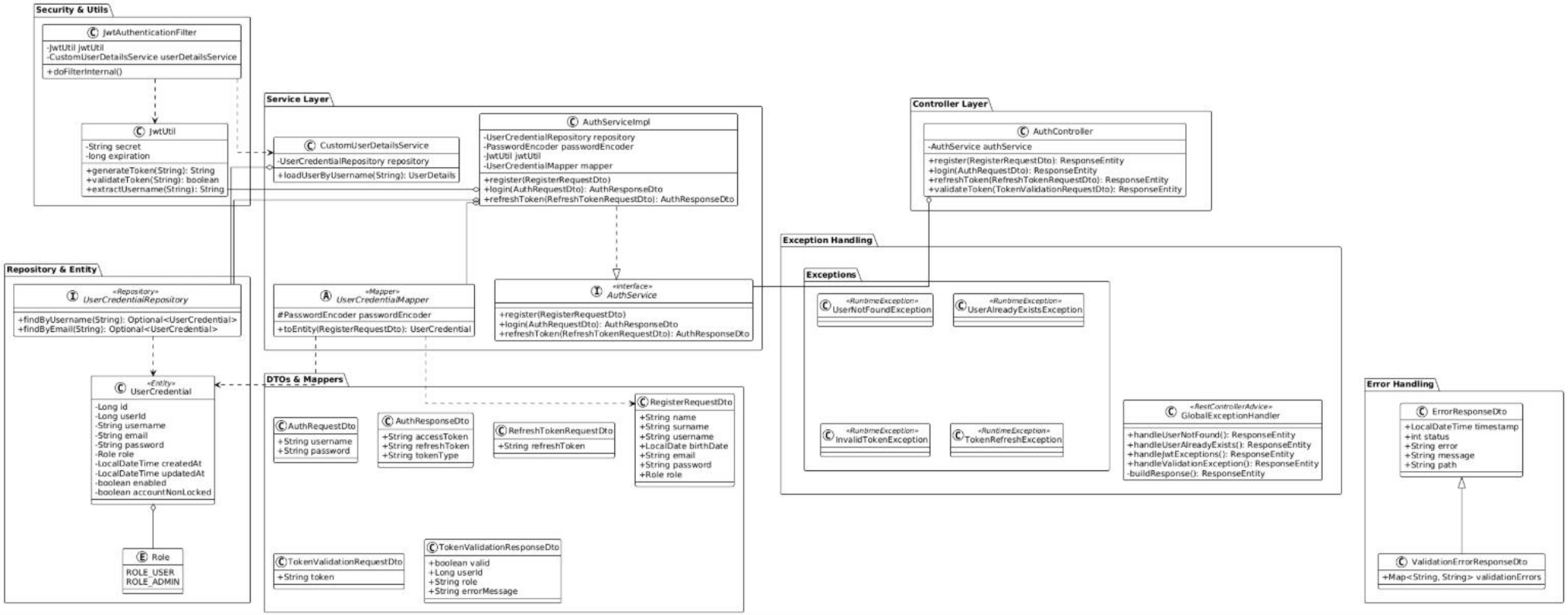
1 Сравнение архитектур:

а) диаграмма классов (As is): Отражает внутреннюю структуру микросервиса авторизации на языке Java (Spring Boot) с детализацией до методов и полей классов.

б) диаграмма компонентов (To be): Отражает распределенную систему, состоящую из множества сервисов (Auth, User, Order, Payment), взаимодействующих через API Gateway и брокер сообщений Kafka.

2 Отличия и причины их возникновения:

Диаграмма классов и диаграмма компонентов отображают проект на разных масштабах: первая отображает детальную структуру одного из микросервисов, а вторая – всю систему целиком.


Security & Utils
JwtAuthenticationFilter
-JwtUtil jwtUtil
-CustomUserDetailsService userDetailsService
+doFilterInternal()
JwtUtil
-String secret
-long expiration
+generateToken(String): String
+validateToken(String): boolean
+extractUsername(String): String
Service Layer
CustomUserDetailsService
-UserCredentialRepository repository
+loadUserByUsername(String): UserDetails
AuthServiceImpl
-UserCredentialRepository repository
-PasswordEncoder passwordEncoder
-JwtUtil jwtUtil
-UserCredentialMapper mapper
+register(RegisterRequestDto)
+login(AuthRequestDto): AuthResponseDto
+refreshToken(RefreshTokenRequestDto): AuthResponseDto
Controller Layer
AuthController
-AuthService authService
+register(RegisterRequestDto): ResponseEntity
+login(AuthRequestDto): ResponseEntity
+refreshToken(RefreshTokenRequestDto): ResponseEntity
+validateToken(TokenValidationRequestDto): ResponseEntity
Repository & Entity
«Repository»
UserCredentialRepository
+findByUsername(String): Optional<UserCredential>
+findByEmail(String): Optional<UserCredential>
«Mapper»
UserCredentialMapper
#PasswordEncoder passwordEncoder
+toEntity(RegisterRequestDto): UserCredential
«interface»
AuthService
+register(RegisterRequestDto)
+login(AuthRequestDto): AuthResponseDto
+refreshToken(RefreshTokenRequestDto): AuthResponseDto
Exception Handling
Exceptions
«RuntimeException»
UserNotFoundException
«RuntimeException»
UserAlreadyExistsException
«RuntimeException»
InvalidTokenException
«RuntimeException»
TokenRefreshException
«RestControllerAdvice»
GlobalExceptionHandler
+handleUserNotFound(): ResponseEntity
+handleUserAlreadyExists(): ResponseEntity
+handleJwtExceptions(): ResponseEntity
+handleValidationException(): ResponseEntity
-buildResponse(): ResponseEntity
«Entity»
UserCredential
-Long id
-Long userId
-String username
-String email
-String password
-Role role
-LocalDateTime createdAt
-LocalDateTime updatedAt
-boolean enabled
-boolean accountNonLocked
Role
ROLE_USER
ROLE_ADMIN
DTOs & Mappers
AuthRequestDto
+String username
+String password
AuthResponseDto
+String accessToken
+String refreshToken
+String tokenType
RefreshTokenRequestDto
+String refreshToken
RegisterRequestDto
+String name
+String surname
+String username
+LocalDate birthDate
+String email
+String password
+Role role
TokenValidationRequestDto
+String token
TokenValidationResponseDto
+boolean valid
+Long userId
+String role
+String errorMessage
Error Handling
ErrorResponseDto
+LocalDateTime timestamp
+int status
+String error
+String message
+String path
ValidationErrorResponseDto
+Map<String, String> validationErrors